DressApp Complete Technical User Manual

Comprehensive user manual and technical reference guide for the DressApp personal wardrobe ecosystem, styling engine, circular marketplace, and administration panels.

---

1. Overview & Technology Stack

DressApp is an AI-driven personal wardrobe manager, styling advisor, and circular marketplace. It helps users manage clothing items digitally, auto-crop and tag them, receive weather- and calendar-aware outfit recommendations, scan EU Digital Product Passports (DPP), and trade garments.

Core Value Proposition

- **Digital Wardrobe Ingestion**: Snapshot or upload photo processing with automated background removal, clothing categorization, and attribute tag generation.
- **AI Virtual Stylist**: A conversational agent that contextually reviews your wardrobe, Google Calendar events, and local weather forecasts to suggest daily outfits.
- **Circular Marketplace**: Secure peer-to-peer buying, selling, swapping, and renting of clothes to reduce fast fashion waste.
- **Cost-Per-Wear (CPW) Analytics**: Insights into wardrobe capitalization value, utilization rates, and usage optimization.

Technology Architecture

- **Monorepo Structure**: Turborepo workspace linking applications (`apps/web`, `apps/mobile`, `apps/android-twa`) and shared libraries (`packages/api-client`, `packages/i18n`, `packages/types`).
- **Backend Edge**: Python 3.11 with FastAPI, using asynchronous Motor drivers connected to a dedicated on-premises MongoDB 7.0 database on NVMe storage.
- **Frontend SPA & Mobile**: **Web**: React 19 single-page application utilizing `useSyncExternalStore` custom stores (`stylistStore`, `dailySuggestionsStore`, `useOutfitStore`, `useClosetStore`, `useSuitcaseStore`), Tailwind CSS, Shadcn/UI primitives, and `react-i18next` supporting 13 localized languages. **Mobile**: Expo SDK 53 / React Native 0.79 with React Navigation Native Stack, NativeWind styling, and offline SWR caching via `MobileClosetRepository`.
- **State & Network Optimization**: In-flight request deduplication, 15-minute store caching, and `visibilitychange` tab revalidation yielding zero background GET requests when idle.
- **Local Machine Learning & Sizing**: CPU-local U2-Net (`rembg`) background matting, SegFormer-b2 clothing parsing, Fashion-CLIP embeddings, and ANSUR II physical body measurement regression model (`body_predictor.py`). Includes the dedicated on-premises `dressapp-eyes` container running a fine-tuned `Qwen2.5-VL-3B-Instruct` model (`Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf` + `mmproj`) via `llama-server` on port 7860 (~2.8 GB RAM) for Free Tier users and automated quota fallback.
- **Conversational STT/TTS**: Real-time client-side Web Speech recognition fallback, multimodal server-side Gemini modulations, and on-device offline Piper/Sherpa-ONNX engines.
- **External Integration Services**: OpenWeatherMap API for weather fetching, Google Calendar OAuth for daily schedule exports, OpenStreetMap (Nominatim) address autocomplete, and PayPal

Subscriptions/Checkout REST APIs.

- **Production Host**: Hetzner Cloud CPX32 VPS (4 AMD vCPUs, 8 GB RAM) at `dressapp.co`, running a 5-container Docker Compose stack (`dressapp-mongo`, `dressapp-backend`, `dressapp-eyes`, `dressapp-frontend`, `caddy`) with Caddy 2 automated TLS reverse proxying.

---

2. Prerequisites

Host Environment Requirements

- **Hardware**: Minimum 8 GB RAM VPS (e.g., Hetzner CPX32 hosting the production `dressapp.co`).
- **Dependencies**: Docker & Docker Compose stack (including on-premises MongoDB 7.0, backend, eyes inference server, frontend, and Caddy TLS termination).
- **Environment Variables**: API keys configuration (`GEMINI_API_KEY`, `EYES_API_TOKEN`, `DEEPGRAM_API_KEY`, `OPENWEATHER_API_KEY`, `PAYPAL_LIVE_CLIENT_ID/SECRET`, and Google Calendar OAuth tokens).

User App Requirements

- **Web Browser**: Google Chrome or Apple Safari (required for full voice feature compatibility).
- **Permissions**: Grant Camera permission (for clothing snapshots and QR scans), Microphone permission (for voice styling), and Location permission (for local weather styling).
- **Zero-BYOK Tiered AI Architecture**: BYOK (user-supplied API keys) is retired; no external API key is ever required from users: **Free Tier**: Uses DressApp's built-in on-premises **Eyes** engine (currently a fine-tuned `Qwen 2.5 VL 3B` model) out of the box. **Manager & Professional Plans**: Enjoy priority routing to the fast Cloud AI model (currently **Google Gemini 3.5 Flash**).

---

3. Step-by-Step Instructions

3.1 Ingesting Garments (Adding Items)

INGESTION PARADIGMS: Photography, EU Product Passports, and Digital Commerce Receipts.

1. Navigate to the **Add Item** screen.
2. Select **Take Photo** (launches native mobile camera) or click **Upload Photos** (opens OS file chooser). You can upload multiple photos with mixed characteristics in a single batch—combining single isolated garments, head-to-toe outfits worn by a person or mannequin, and unworn piles/clusters of garments—without selecting any manual pipeline mode tabs.
3. The client computes each image's SHA-256 and horizontal difference-hash (dHash) in-browser (~100-180ms) to check against your existing closet.
4. If a match is found, the **Duplicate Preflight Dialog** opens showing matching previews. Select **Skip** or **Add anyway**.
5. **Automatic 3-Pipeline Photo Routing (`detect_photo_pipeline`)**: Once accepted, the backend inspects SegFormer semantic masks (`face_skin_ratio`, `anatomical_stack`, connected

component topology) and automatically routes each photo to the optimal pipeline: `single` **Pipeline**: Single isolated garment matted via studio alpha blending (`rembg` + SegFormer mask intersection). `multi` **Pipeline**: Multi-garment outfit worn by a model or mannequin (`face_skin_ratio >= 0.008` or vertically aligned `anatomical_stack`), isolating upper-body, lower-body, dress, outerwear, footwear, and accessory crops while filtering exposed skin. `mass` **Pipeline**: Unworn garment clusters, flat-lay arrangements, or multiple garments hanging on wall hooks (`parse_unworn_cluster`), separating disconnected components and running semantic-color watershed splitting on touching multi-color items while protecting screen-printed chest graphics.

6. The server starts an NDJSON stream (`detect`, `field`, `item`, `done`), displaying cropped preview frames within seconds and allowing you to edit item details immediately while the backend completes tagging—including deterministic **Sweatshirts & Hoodies** vs. **T-Shirt** classification and 60+ **Traditional & Cultural** attire terms (`Traditional` dress code and localized heritage tags across all 13 languages).
7. **AI Quality Checker & Automatic Generative Repair**: The visual Quality Checker inspects each cropped garment for occlusions (bags, arms, hair) and camera frame cutoffs: **Complete**: Intact, unoccluded garments are matted directly. **Image Completion**: If an item has missing side contours, occluded sections, or clipped hems/collars, `gemini-3.1-flash-lite-image` (Nano Banana) automatically outpaints and completes the missing fabric into a full, symmetrical piece. **Full Studio Reconstruction**: Severely cut-off items (such as shoes showing only toe caps) are fully regenerated into pristine studio catalog photographs.
8. Verify auto-detected tags (color, fabric, fit, pattern, dress code, occasion). If the cutout shape is incorrect, change the **Category** dropdown; this triggers SegFormer to automatically re-crop the garment.
9. Click **Save** to optimistically paint the item in the closet grid immediately (~16ms) while background generative reconstructions and WebP thumbnails conclude in the background.

1. Tap the **Scan QR (DPP)** button on the Add Item page.
2. Grant camera permissions and align the QR code printed on the garment's retail label, or upload a saved QR screenshot.
3. The backend resolves the URL and executes SSRF safety checks (blocking private IP ranges).
4. The system scrapes the JSON-LD schemas to extract brand, materials composition, supply-chain trace, carbon footprint, and care guidelines.
5. Review the extracted data displayed in the green **Verified DPP Data** accordion panel and click **Save**.

1. **Digital Import Interface**: Open the **Digital Import** tab on the Add Item page.
2. **Sub-mode Selection & Processing**: **Paste Text**: Paste raw email bodies, transactional invoices, or copy-pasted order receipts. This text is sent directly to Gemini multimodal vision models for processing. **Upload Image**: Upload invoice/receipt screenshots or photos. For multi-item receipts, users can drag and resize bounding box crop selectors (`[{id, x, y, w, h}]` represented as container percentages) to crop individual items. An off-screen canvas exports a 90% quality JPEG Blob of the selected region to submit to the backend. The backend executes dual-path concurrent processing: multimodal vision/OCR for transaction details, and SegFormer classification for garment characteristics (color, shape, pattern). **Upload PDF**: Submit store PDF invoices (e.g. Zara or ASOS). The backend uses Gemini vision OCR with column-merging logic to parse items, falling back to local `pypdf.PdfReader` text extraction if offline. Bounding selectors allow line-range cropping on the text output before sending it to the parser. **Web Link**: Input order confirmation links. The backend fetches pages via `httpx` using a browser-like User-Agent and a 15-second timeout safeguard, routing them by content-type (HTML page text parsing vs image vision processing).
3. **Data Merging Rule**: Transaction details (price, brand, size, quantity) are extracted from OCR, taking precedence over vision model inference. Visual attributes (clothing category, secondary colors) are classified by SegFormer.

4. **Receipt-Locked Fields & Saving**: All confirmed details are saved in the database with `from_receipt: true` and their field names recorded in `receipt_locked_fields`. Any subsequent visual re-analysis (e.g. when the user updates the item thumbnail) will never overwrite fields protected in this list. Click **Save** to confirm.

1. **Interactive Prompt Box**: Open any garment in your closet to access the **Item Details** editor, and locate the **Re-analyse Photo & AI Eyes Assistant** card.
2. **Conversational Instructions**: Type natural language requests or tap the microphone icon to dictate prompt instructions: *"Remove the shoes" "Complete the hole where the hand was" "Remove the metal studs from the jacket's front" "Refine fabric composition to 100% cashmere"*
3. **Quick Prompt Chips**: Tap quick starter chips (■ *Remove shoes*, ⤙■ *Complete hole*, ■ *Remove studs*, ■ *Refine fabric*) for instant 1-tap edits.
4. **Nano Banana Inpainting (`gemini-3.1-flash-lite-image`)**: The multimodal decision engine classifies visual edit requests and executes targeted inpainting against the original/segmented pixels. Nano Banana reconstructs the missing or occluded regions, rendering a photorealistic preview card directly in the chat thread.
5. **Interactive Application**: Tap **Apply as garment photo** to update the in-memory garment photo, then click **Save** in the action bar to persist the new image to MongoDB.

1. **Activate Batch Selection**: Open the **Closet** screen and tap the **Select / Batch Edit** button in the header toolbar.
2. **Select Multiple Garments**: Tap checkboxes on garment cards to select items in bulk, or use category quick-select pills (*"Select all Tops"*, *"Select all Outerwear"*).
3. **Open Group Tagging Sheet**: Tap **Group Tag** in the bottom floating actions bar.
4. **Batch Assign Attributes**: **Season**: Bulk assign seasonal suitability (*Spring*, *Summer*, *Fall*, *Winter*). **Formality & Dress Code**: Apply unified style classifications (*Casual*, *Smart-Casual*, *Business*, *Formal*, *Athletic*, *Loungewear*). **Occasion & Tags**: Add custom organizational tags (e.g., `"Workwear"`, `"Vacation"`, `"Capsule Wardrobe"`).
5. **Atomic Batch Application**: Click **Apply Tags to N Items**. The frontend submits an atomic batch update, instantaneously synchronizing your closet store via `useSyncExternalStore` (~16ms) and immediately enhancing AI Stylist context matching without manual per-item editing.

3.2 Conversational AI Virtual Stylist

Describe styling dilemmas and receive hands-free spoken outfit advice.

1. Navigate to the **AI Stylist** screen.
2. Click the microphone icon `[Microphone]` in the chat input bar or type your styling question directly.
3. Speak your request (e.g., "What top matches my beige trousers for a rainy outdoor lunch?").
4. If Web Speech is supported, your voice transcribes live in the input box. If not, the app records a WebM file and uploads it.
5. **Multi-Tier Brain Resolution**: **Manager & Professional Plans — Fast Cloud Model (Google Gemini 3.5 Flash)**: Paid subscribers on the **Manager** and **Professional** plans route to **Google Gemini 3.5 Flash** (`gemini-3.5-flash-lite`) via `llm_gateway.py`, delivering sub-350ms initial token latency, natural conversational advice, and precise JSON schema matching across complex wardrobe inventories. **Free Tier — On-Premises VPS Eyes (Fine-Tuned `Qwen 2.5 VL 3B`)**: Free-tier users are

served by DressApp's self-hosted, fine-tuned **Qwen 2.5 VL 3B** engine running in `dressapp-eyes` on port 7860. **Transparent Quota Fallback**: If external cloud API limits or quota errors (`429` / `RESOURCE_EXHAUSTED`) occur, the backend seamlessly falls back to on-premises Eyes (`Qwen 2.5 VL 3B`). An informative banner (*"Using Platform Stylist (Quota Fallback)"*) appears above the assistant's reply so your styling session is never interrupted by a 500 error.

6. The stylist presents head-to-toe matching outfit cards with detailed styling rationales.
7. Spoken audio advice plays automatically using preselected voice profiles (`puck`, `aoede`, or `charon`).
8. Tap **Play reply** (or **Replay** in RTL mode) on the card to replay the voice audio anytime.

3.3 Profile, Preferences, and Subsystem Dependencies

The Profile page serves as the core control panel for DressApp. Configuration fields directly impact the performance, routing, and behavior of downstream modules.

1. **Photos & Digital Avatar Stage (`AvatarViewer2D` & `DynamicAvatar`) Why does it matter?**: Renders your visual identity across all try-on canvases using a dual-mode stage (segmented real-body photo cutout vs dynamic 2D Bezier vector SVG mannequin). **Subsystem Dependencies**: Photo cutouts are background-matted via local U2-Net (`rembg`) and downscaled in-browser to a maximum of 1280px at 82% quality to fit within MongoDB's 16MB document ceiling. The stage applies calibrated positional landmarks (`top-[14.5%]` collar-to-neckline, `top-[36.5%]` waistband-to-waistline, `bottom-[2%]` footwear plane) and proportional chest/hip scaling ($scaleX$). Click *Remove Photo* to instantly switch back to the 2D SVG vector mannequin.
2. **Style Profile (Modest rules, Dress code) Why does it matter?**: It establishes personal boundaries for recommended outfits, preventing the AI from generating inappropriate style suggestions. **Subsystem Dependencies**: The selected parameters (e.g., Modest clothing constraints) are fed directly into the styling prompts for Gemini 2.5 Flash, filtering matching wardrobe outputs before they are shown.
3. **Details (Name, Phone, Occupation) Why does it matter?**: It customizes the communication tone and routes notification alerts. **Subsystem Dependencies**: The user's name is dynamically parsed into emails and system-level pushes. The phone number serves as a fallback registry for scheduled alerts. The occupation parameter is fed to the stylist LLM and Trend Scout personalization ranker to customize proposals.
4. **Body Measurements & Sizing (ANSUR II Regression Model & Sizing Predictor) Why does it matter?**: It eliminates sizing guesswork, allowing automatic retail size calculation, external retail size comparison, and accurate virtual layering. **Subsystem Dependencies**: Entering 4 basic parameters (**Height**, **Weight**, **Waist**, **Foot Length**) triggers the scikit-learn ANSUR II regression model (`body_predictor.py`) to auto-predict 6 structural dimensions (*Shoulders*, *Chest*, *Hip*, *Sleeve*, *Inseam*, *Outseam*). **Deterministic Size Translation**: Once continuous measurements are predicted, the backend sizing engine dynamically converts them into retail clothing sizes: **Shirt Size** (XS-XXL based on chest), **Pants Size** (Waist in inches), **Shoe Size** (US Men/Women and EU standards based on foot length and gender), **Dress Size** (US 0-14+ based on chest, waist, and hips), and **Bra Size** (Band + Cup based on chest and estimated underbust). **Full Auto-Population**: These recommended retail sizes are automatically populated in the *Detailed Edit Mode* fields inside the profile dashboard, filling all 15+ sizing parameters down to shoe size without manual entry. **Integrations**: Measurements are queried directly by the **Shopping Assistant** Chrome Extension content scripts to read size tables on partner websites (Zara, Asos) and recommend sizes.
5. **Lifestyle (Status, Sex) Why does it matter?**: It tailors default recommendations and scores content algorithms. **Subsystem Dependencies**: The sex selection directly affects the ranking logic of daily

Trend Scout cards. If a news card category mismatches the user's sex, the algorithm applies a -2.0 score penalty, demoting it in the feed.

6. **AI Configuration & Subscription Plans (Tiered Model Routing, Quota Fallback, Credits) Why does it matter?**: It governs LLM brain routing, subscription tiers, and credit balances (with zero BYOK configuration required). **Subsystem Dependencies**: **Manager & Professional Plans (Fast Cloud Model)**: Subscribers on the **Manager** and **Professional** plans enjoy priority routing to the fast Cloud AI model (currently **Google Gemini 3.5 Flash** / `gemini-3.5-flash-lite`) across all core styling, migration, and planning flows via `llm_gateway.py`. **Free Tier (On-Premises VPS Eyes)**: Free-tier users are powered by DressApp's on-premises **Eyes** engine (currently a fine-tuned **Qwen 2.5 VL 3B** model) at zero variable cloud cost. **Automated Quota Fallback**: If external cloud calls encounter rate limits (`429`), `RESOURCE_EXHAUSTED`, or upstream timeouts, `FallbackBrain` immediately redirects the request to on-premises Eyes (`Qwen 2.5 VL 3B`), rendering a non-blocking notification banner (`stylist.fallbackQuotaBanner`) without interrupting the styling conversation. **Trial & Paid Plans**: Free baseline, Manager tier, and Professional tier. If credits are exhausted, the app enters an async pause-and-resume wait state (up to 60s) checking for top-up events.
7. **Scheduler & Push (Frequency, daily alarm, style focus) Why does it matter?**: It manages automatic daily style pushes. **Subsystem Dependencies**: Activates `APScheduler` cron jobs on the FastAPI backend. Every morning, it triggers push notifications via `pywebpush` using the client's VAPID keys, matching the selected style focus parameters.
8. **Google Calendar (OAuth sync, export rules) Why does it matter?**: It bridges your wardrobe directly with your real-world calendar events. **Subsystem Dependencies**: Authenticates via Google OAuth. The scheduler queries your calendar to identify events, formats outfits, and pushes calendar events straight to your Google Calendar agenda.
9. **Location Services (GPS tracking, weather accuracy) Why does it matter?**: It coordinates weather-appropriate suggestions and local transaction radius filters. **Subsystem Dependencies**: Triggers `navigator.geolocation` reverse-geocoding. Coordinates are sent to OpenWeatherMap API to adjust the stylist recommendations (e.g., rainwear for downpours). It also calculates distances for local Marketplace listings and experts (e.g.,Lisbon radius checks).
10. **Voice & Language (Virtual stylist voice selection) Why does it matter?**: It establishes text dictionary locales and voice modulations. **Subsystem Dependencies**: Controls the active language for translations via `react-i18next`. The voice selection maps BCP-47 voice codes (e.g., `he-IL` or `ar-JO`) to client Web Speech synthesis voices or offline Piper TTS models.
11. **Invite Friends (Share payload API) Why does it matter?**: It provides a viral loop for free closet expansion. **Subsystem Dependencies**: Appends the referrer's MongoDB ID to the URL. New registrations dynamically query this ID and atomically increment the referrer's `closet_capacity_bonus` by +10 slots, modifying the limit guards in `closet.py`. Free tier capacity is capped at 50 items baseline, but can expand up to a maximum limit of 150 items through referral credits (+10 slots per referral).
12. **Developer & Administrator Access (Google OAuth) Why does it matter?**: Ensures privileged platform governance and diagnostic tools are restricted to verified accounts. **Subsystem Dependencies**: Gated through Google OAuth (`dressapdeveloper@gmail.com`). Users defined under `ADMIN_EMAILS` in production environment settings are assigned administrative roles (`role: "admin"`).

3.4 Wardrobe Migration from Competitors

DressApp provides a streamlined way to migrate your wardrobe from popular competitor applications, eliminating the need to re-photograph every item.

Supported Competitor Apps:

- Whering
- Acloset
- Stylebook
- Smartli
- BeautyAI
- Other (custom input)

Detection mechanism: `'ontouchstart' in window` identifies touch-primary devices (phones, tablets).

Entry point gating — Four locations in the SPA check this flag before allowing migration:

Entry Point	File	Desktop Behavior	Mobile Behavior
Auto-trigger (new user)	`AppLayout.jsx`	Renders migration modal	Shows toast.info (8s)
"Import Wardrobe" pill	`Profile.jsx`	Opens migration modal	Shows toast.info (8s)
Shopping Assistant section	`Profile.jsx`	Renders draggable bookmarklet link	Shows info box with desktop guidance
"Migrate" button	`LoginClosetReminderModal.jsx`	Opens migration modal	Shows toast.info (8s)

Mobile guidance message: "Wardrobe import is available on the desktop version of DressApp. Please open your account on a desktop browser to continue."

Rationale: The `getDisplayMedia` API (required for tab-level screen capture) is not supported on mobile browsers. The bookmarklet installation flow (drag to bookmarks bar) is also desktop-only.

The bookmarklet is a self-contained `async` IIFE encoded as a `javascript:` URI that runs on the competitor's page.

Initialization Phase:

1. Remove Chrome extension widgets that conflict with migration
2. Send message to DressApp Chrome extension
3. Inject heartbeat animation CSS
4. Create fixed overlay widget with "Share & Start Agent" button

Screen Capture Phase:

1. Request tab-level screen share via `navigator.mediaDevices.getDisplayMedia()`
2. Primary: `{ video: { displaySurface: 'browser' }, preferCurrentTab: true }`
3. Fallback: `{ video: true, preferCurrentTab: true }`
4. Validates tab-level capture (rejects "window" or "screen" share modes)
5. Hidden `<video>` element receives MediaStream, waits for readyState >= 3

Card Detection (Dual-Strategy DOM Scanner):

Strategy 1 — `<img>` candidates:

- Query all `<img>` elements on the page
- Filter: minimum 40×40px, within viewport bounds, not inside `<header>`, `<nav>`, or `<footer>`
- Walk up DOM to find card-shaped ancestor (width 80–600px, height 80–800px, aspect ratio 0.7–2.5)

- Validate image fills at least 50% of card area
- **Fully-visible gate**: `r.top >= 0 && r.bottom <= innerHeight` — rejects partially clipped cards

Strategy 2 — `<div>` fallback:

- When no `<img>` cards found, scan `<div>`, `<li>`, `<a>`, `<section>` elements for card-shaped containers with CSS `background-image` or nested `<img>` elements

Deduplication: Coordinate-based rejection — two rects within 20px are considered duplicates. *Row grouping:* Cards grouped by Y-center tolerance of 30px for row-aware scrolling.

Crop Engine (`cropCardFromStream()`):

1. Maps CSS viewport coordinates → video pixel coordinates via `scaleX = videoWidth / innerWidth`
2. Clamps source rectangle to video bounds
3. Draws to off-screen `<canvas>`
4. **Quality gate**: Computes color variance — rejects crops with variance < 2
5. **Aspect ratio gate**: Rejects crops where width > 5× height or vice versa
6. Outputs JPEG base64 at quality 0.85

Autoscroller (Row-Aware Scroll Loop):

1. Hide overlay → wait 300ms + double-rAF (layout settle)
2. Detect fully visible cards via `getVisibleGarmentRects()`
3. For each new card: crop from video stream → store
4. Every 15 cards: postMessage(STREAM batch) to parent window
5. Show overlay with card count
6. Calculate scroll amount (row-aware, based on grouped row heights)
7. Scroll, wait 400ms + double-rAF
8. Center the middle row on screen
9. Adaptive lazy-image poll (12s timeout, 600ms intervals)
10. Terminate when: no change + no new rects + noChangeCount >= 3

Completion Phase:

1. Stop MediaStream tracks (release screen capture)
2. Send remainder cards via STREAM message
3. Send DRESSAPP_MIGRATION_COMPLETE with total_cards + app_name
4. Clear harvestedCards array (memory cleanup)
5. Show green completion badge in overlay

A global React component mounted at the App level that survives route navigation.

Message Protocol:

Message Type	Payload	Behavior
`DRESSAPP_MIGRATION_STREAM`	`{ cards: [{crop_base64}] }`	Appends to `collectedCards[]` buffer
`DRESSAPP_MIGRATION_COMPLETE`	`{ total_cards, app_name }`	Saves to DB + starts Stylist worker

Data Flow (Atomic Single-Call Pipeline):

1. Collect cards from STREAM messages → `collectedCards[]`

2. On COMPLETE: Filter: `collectedCards.filter(c => c.crop_base64)` Clear buffer: `collectedCards.length = 0` Single atomic call: `api.saveMigrationCrops({app_name, cards})` Navigate: `navigate('/closet')`

The core production endpoint that atomically saves crop images to MongoDB and kicks off Stylist analysis in a single request.

Request:

```
class MigrationSaveCropsIn(BaseModel):
    app_name: str = "Competitor App"
    cards: list[dict[str, Any]]  # Each must have crop_base64 (base64 JPEG)
```

Processing:

1. For each card: Extract crop_base64 (raw base64 or data URL with prefix) Decode to raw bytes Validate minimum size (>100 bytes) Validate aspect ratio (width/height between 0.2 and 5.0) Build data URL for segmented_image_url Create ClosetItem (brand=app_name, source="Private", category="Top") Compute source_phash and source_color_sig via image_hash service Insert into MongoDB
2. If items saved AND garment_vision_service configured: Snapshot saved items by ID list (avoids race condition) Spawn `asyncio.create_task(_run_reanalyze_items)` Return job_id for status tracking
3. Return `{items_saved, items_skipped, item_ids, app_name, job_id}`

Response:

```
{
  "items_saved": 23,
  "items_skipped": 2,
  "item_ids": ["item_a1b2c3d4e5f6", ...],
  "app_name": "Whering",
  "job_id": "reanalyze_a1b2c3d4e5f6"
}
```

A shared async function that processes closet items through Stylist (Gemini) analysis. Runs as an `asyncio.create_task` — completely independent of the client.

Processing Per Item:

1. Resolve best image URL (priority chain): segmented_image_url → cutout_url → clean_image_url → image_variants.webp.large → image_variants.webp.medium → reconstructed_image_url → image_variants.original → original_image_url → image_url
2. Download image bytes via `_read_image_bytes_from_url()`
3. Run Gemini analysis under `_ANALYZE_LOCK` (semaphore=1)
4. Apply `_safe_analysis()` normalization
5. Filter unidentifiable items via `_is_unidentifiable()`
6. Build update_doc with Stylist fields: title, name, caption, category, sub_category, item_type, brand, gender, dress_code, season, tradition, colors, fabric_materials, pattern, state, condition, quality, repair_advice, tags
7. Mirror dominant color/material into legacy scalars
8. Update MongoDB via `repos.update()`

Standalone endpoint for manually re-triggering Stylist analysis on all items matching a brand.

```
class MigrationReanalyzeIn(BaseModel):
    app_name: str = "Competitor App"
```

The modal UI that guides users through the migration flow:

1. **Ask step** (`step === 'ask'`): Welcome dialog with "No, Start Fresh" / "Yes, Import Closet"
2. **App search step** (`step === 'app_search'`): Preset grid (Whering, Acloset, Stylebook, Smartli, BeautyAI) with manual app name input
3. **Web login step** (`step === 'web_login'`): Bookmarklet install instructions, "Import wardrobe" button opens popup via `window.open(targetLoginUrl, '_blank')`

Mobile/Desktop Branching:

- **Desktop**: Renders a draggable `<a>` element with `href="javascript:..."` — user drags to bookmarks bar
- **Mobile**: Renders a hidden `<textarea>` with the bookmarklet code, uses `document.execCommand('copy')` to copy to clipboard

A `useSyncExternalStore`-compatible global store that tracks:

- **Analyze jobs**: In-flight `/analyze` requests (registered by AddItem.jsx)
- **Polish jobs**: Pending background matte/reconstruction tasks

The global poller runs at 3s intervals and checks polish pending items via `api.getItem(id)` with a 5-minute timeout per item.

A draggable glass-morphism pill positioned at `bottom-20` on mobile and `bottom-4` on desktop. Subscribes to `workStore` via `useSyncExternalStore` and renders:

- **Analyzing**: "Analyzing N photos..." with pulsing sparkle icon
- **Done**: Brief "✓ Done" linger (1.2s) before auto-hiding

1. **Access Migration:** From your Profile page, click the **"Import Wardrobe"** button (desktop only)
2. **Select Source:** Choose your competitor app from the grid or enter a custom app name
3. **Login to Competitor:** A new browser tab will open to your competitor's login page
4. **Install Bookmarklet:** Drag the "Share & Start Agent" button to your browser's bookmarks bar
5. **Navigate to Wardrobe:** Go to your wardrobe/closet view in the competitor app
6. **Run Migration:** Click the installed bookmarklet button
7. **Automatic Capture:** The agent will: Scroll through your wardrobe Detect and crop garment images Stream captured items back to DressApp in batches of 15
8. **AI Analysis:** Each captured garment is automatically analyzed by our Stylist AI, which identifies: Item type and category Color palette Fabric/material Brand (where visible) Season and occasion Dress code level
9. **Review & Edit:** Once migration completes, review your imported items in the Closet tab

- Uses `getDisplayMedia` API for tab-level screen capture
- Items are processed server-side, so you can close the competitor tab once migration starts
- Duplicate detection prevents importing the same item twice
- Unidentifiable items (obscured images, non-garment items) are automatically filtered out
- Mobile browsers are not supported for migration (use desktop instead)
- Stylist worker runs server-side — closing browser has no effect once save-crops returns

1. **Bookmarklet card detection**: Requires cards to be fully visible in viewport. Non-standard HTML (CSS backgrounds, canvas, shadow DOM) may not be detected

2. `_is_unidentifiable` **filter**: Drops crops where Gemini returns empty analysis, give-up phrases in title/caption, or missing `item_type` + `sub_category`
3. **Gemini API failures**: Individual `analyze()` calls may fail with quota/timeout errors — items are skipped
4. **Browser closure during bookmarklet crawl**: If DressApp tab is closed while bookmarklet is running, crops are lost
5. **Browser closure after save-crops**: Safe — Stylist worker runs server-side

- If the bookmarklet doesn't appear, ensure your bookmarks bar is visible (Ctrl+Shift+B)
- For best results, ensure all wardrobe items are fully visible in the browser window
- If migration stalls, try refreshing the competitor page and restarting the bookmarklet
- Check the DressApp console for diagnostic messages if items aren't appearing
- **Manual re-trigger:** Use the backend endpoint to re-analyze all items from a specific competitor:`curl -X POST -H "Authorization: Bearer <token>" \ -H "Content-Type: application/json" \ -d '{"app_name": "Whering"}' \ https://dressapp.co/api/v1/closet/migration/reanalyze-by-brand`



```
■■■
```

3.7 Suitcase Assistant

Organize your packing requirements for trips without overpacking.

1. Go to the **Suitcase** page and fill out the Trip Context form (destination, start/end dates, trip category, calendar events).
2. The AI generates a customized packing list and daily outfits based on trip duration and weather forecasts.
3. Review the packing progress. If an important item is missing (e.g., umbrella for rain, swimwear for beach), the system alerts you and suggests matches from the marketplace or local stores.
4. Use the integrated chat box to refine suggestions (e.g., "Change day 2 to casual evening wear"). The assistant edits the suitcase while preserving the rest of the list.
5. Tap **Approve Suitcase** to finalize your plan.

3.8 Scheduler & Push Reminders

Set daily styling alerts to receive outfit recommendations automatically.

1. Open **Profile** and go to **Scheduler & Push**.
2. Toggle notifications, set a daily notification time, weekday frequency, and styling focus theme.
3. Every morning, the background cron task (`APScheduler`) checks weather forecasts and sends a push notification.
4. Tap the notification on your device (or view the web app's Notification Center) to open a proposal dialog displaying 3 styled suggestions.
5. Save a suggestion directly to your **Wardrobe Diary**.

3.9 Trend Scout (Autonomous Fashion Radar & Intelligence)

Discover curated global and domestic fashion trends synchronized with your personal wardrobe.

1. **Seven Curated Channels**: ■ **Local News (`local`)**: Regional fashion events, domestic designer showcases, and boutique news anchored directly to your home country. ■ **Runway (`runway`)**: Haute couture, seasonal fashion weeks (SS/FW lookbooks), directional silhouettes, and major fashion house aesthetics. ■ **Street Style (`street`)**: Urban fashion movements, sneaker releases, utilitarian layering, and modern subcultural aesthetics. ■ **Sustainability (`sustainability`)**: Eco-textile innovations, circular fashion initiatives, zero-waste garment care, and fair-wage fashion. ■ **Influencers & Icons (`influencers`)**: Viral creator styling movements (quiet luxury, old money, Y2K), red-carpet recaps, and contemporary creator formulas. ■■ **Vintage / Archival (`vintage`)**: Archival designer retrospectives, heritage denim curation, and non-commercial second-hand history. ■ **Care & Repairs (`maintenance_repairs`)**: Actionable tutorials on visible mending, fabric depilling, sneaker cleaning, cobbling, and garment longevity.
2. **Dual Gender Ecosystems & Instant Switcher**: Distinct intelligence streams independently curated for Men's and Women's fashion aesthetics. Header ecosystem selector allows instantaneous switching

between Women's and Men's radar without altering account defaults.

3. **Automated Synthesis Engine & Strict Heuristic Shield**: Scheduled cron sweeps (monthly on the 1st at midnight UTC, daily at 07:00 UTC, or on-demand user refresh via ■) crawl authoritative publications via `httpx` + `BeautifulSoup`. Google Gemini synthesizes magazine-grade editorial cards with verified OpenGraph hero images and authentic outbound source articles. Strict heuristic shield actively rejects e-commerce checkout URLs (`/cart`, `/buy`, Shopify/WooCommerce), fast-fashion storefronts, paywalls/login gates, and broken links. Priority 2-tier concurrency localizes cards into all 13 supported languages via Gemini Flash with full RTL support.
4. **Demographic Personalization & Social Sync**: Cards are dynamically re-ranked via demographic scoring considering gender alignment (+15/-10), publication recency (+10), wardrobe silhouette match (+6), and geographic affinity (+8/+3). Configure aesthetic preferences and link social profiles (Instagram, Pinterest, TikTok, Facebook, Threads, X) via the Settings Modal (■■).
5. **Interactive Actions**: **Style with My Closet**: 1-Tap button transfers trend tokens (silhouette, drape, color scheme) to the conversational AI Stylist, instantly generating outfit combinations from clothes you already own. **Bookmark to Wardrobe Diary**: Pin inspirational trends to your personal diary to guide future purchases or custom outfit planning.

3.10 Marketplace (Resale, Renting, Swapping, Gifting)

Engage in the peer-to-peer circular fashion marketplace.

- **Create a Listing**: Open an item's detail page, select **Edit Intent**, and choose a non-private intent: *For Sale*: Input list price and currency (detects your default currency via regional locale preferences). *Rent*: Set daily rental tariff and borrowing conditions. *Swap*: Mark item open for trade. *Donate*: Publish item for free.
- **State Synchronization**: Listings propagate to the feed automatically. The client uses `useSyncExternalStore` and IndexedDB caching to load search parameters with zero-latency.
- **Try-On Sandbox**: Renters/buyers can test-fit a listing against items in their private closet before checking out.
- **Transactional Checkout**: *Purchasing/Renting*: Complete transaction via integrated PayPal buttons. Captured webhooks notify the seller, flip listing state to sold/rented, and log transactions in the ledger minus the 7% platform fee. *Bartering (Swapping)*: Prospective swappers propose trades. Lister receives confirmation emails to accept or decline.

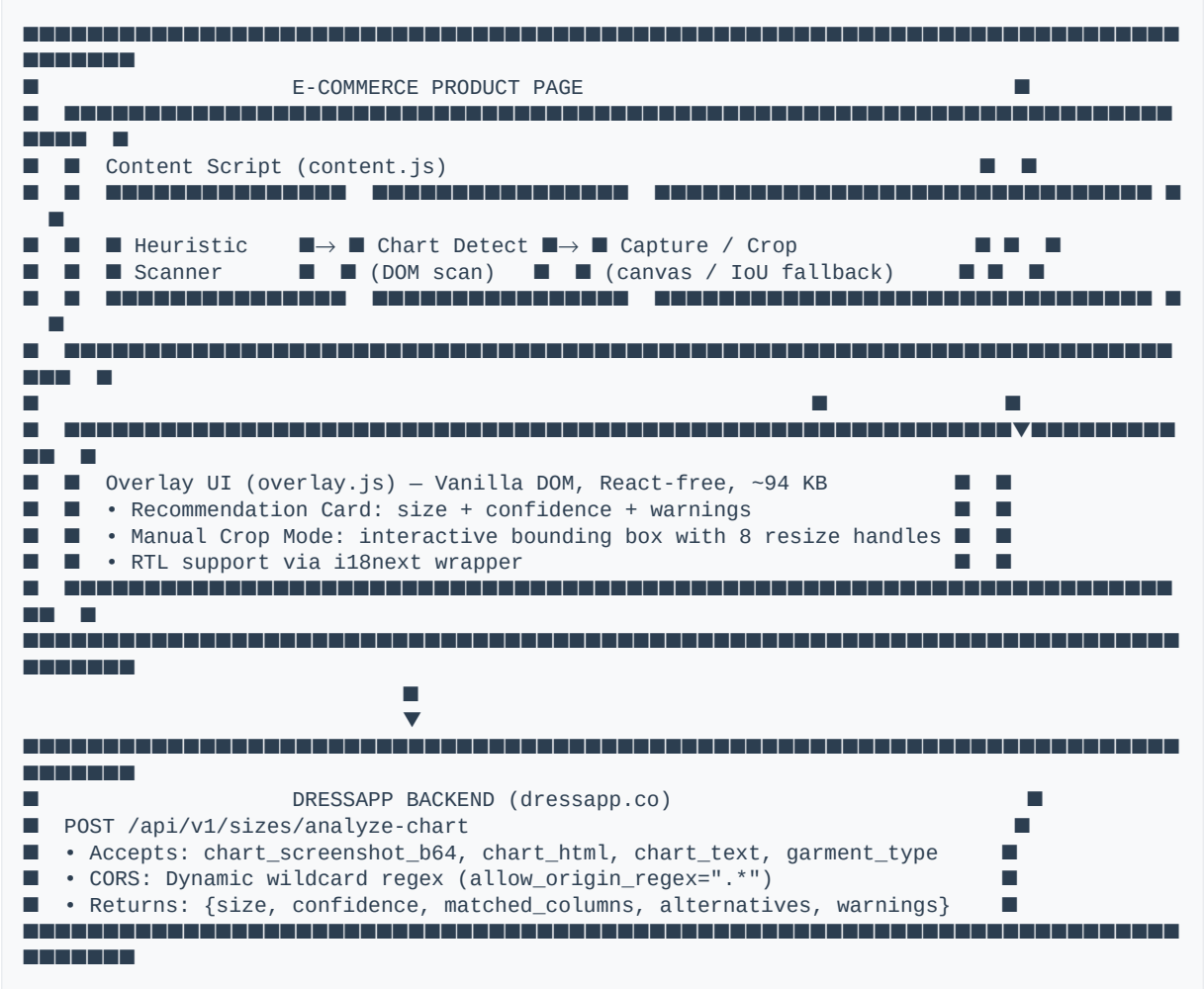
3.11 Shopping Assistant Chrome Extension & Mobile Bookmarklet

A cross-platform client-side utility that resolves sizing uncertainty on third-party e-commerce storefronts by overlaying personalized size recommendations directly on product pages.

The DressApp Extension Ecosystem operates in two modes:

Mode	Runtime	Screenshot API	DOM Extraction	Storage
Chrome Extension (MV3)	Background Service Worker	`chrome.tabs.captureVisibleTab`	Direct DOM access	`chrome.storage.local`
Mobile Bookmarklet	Page sandbox (IIFE)	Not available	IoU-based DOM fallback	`localStorage` with `dressapp_widget_` prefix

Dual-Mode Routing Architecture:



Multi-Step Heuristic Scoring Algorithm:

1. HTML Table Scanner:

- Iterates over all `<table>` elements in the DOM
- **Base Checks**: Must contain at least 1 number and be longer than 40 characters
- **Keyword Hits**: Adds `+8` points for every matching measurement string (e.g., `bust`, `waist`, `inseam`)
- **Row Count Boost**: Adds `+5` points per row (capped at `+50`)
- *Threshold*: Tables scoring ≥ 30 are selected

2. Image Chart Scanner:

- Evaluates all visible `<img>` elements based on layout and metadata rules
- **Aspect Ratio**: Rejects images with ratio < 0.4 or > 4.0 (typically thin side-banners)
- **Area Limit**: Must be at least 160 × 120 px
- **Semantic Boost**: Adds `+30` points for alt texts or aria labels containing strong keywords (e.g., `"size chart"`)
- **Product Photo Penalty**: Deducts `-14` points if parent classes match typical image gallery keywords (`gallery`, `carousel`, `swiper`)

3. Bounding-Box DOM Fallback (IoU Matcher):

- When running as a bookmarklet (no access to extension screenshot APIs), the manual crop draws a CSS-pixel box

- Executes an **Intersection over Union (IoU)** search over the DOM: $$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$
- The element yielding the highest IoU score is selected as the target container
- If this container points to an `<iframe>` on the same domain, automatically extracts the inner `contentDocument.body` contents

1. Auth Handoff (`RECEIVE_HANDOFF`): When a user clicks "Connect to DressApp" on the main portal, the portal broadcasts a custom event:

```
window.postMessage({ type: 'DRESSAPP_EXT_TOKEN', token: '...', backend: '...' }, '*');
```

The content script intercepts this message and forwards it via `sendToBackground` to be securely saved inside Chrome's local storage (or local storage namespaces prefixed with `dressapp_widget_` if running in bookmarklet mode).

2. CORS Bypass Strategy: The FastAPI backend allows arbitrary origins using a dynamic wildcard regex (`allow_origin_regex=".*"`). This allows bookmarklets running on external store origins like Shein to communicate directly with `https://dressapp.co/api/v1` without triggering preflight blocks.

3. Payload Construction: The endpoint `/sizes/analyze-chart` consumes a unified payload:

```
interface AnalyzeChartIn {
  chart_screenshot_b64?: string; // Crop image from canvas
  chart_html?: string;           // Extracted DOM html fallback
  chart_text?: string;           // Extracted raw innerText fallback
  garment_type: string;          // Extracted type hint
  store: string;                 // Domain name
  page_url: string;              // Full URL
  page_title: string;            // Page header
}
```

- **React-Free Footprint**: The overlay UI (`overlay.js`) compiles into standard vanilla DOM creation calls, isolating variables and preventing CSS and JS namespace leakage from crashing parent page scripts
- **Visual Isolation**: All modal elements styled using custom classes mapped inside `content.css` with high-specificity selectors to prevent host page CSS frameworks (Bootstrap, Tailwind) from overriding the widget
- **Dynamic Internationalization**: Custom wrapper around `i18next` that parses document headers to detect Hebrew and Arabic locales, automatically inserting `dir="rtl"` flags to flip layouts and mirror warning elements

Autodetect Flow:

1. Upon page load, the mutation observer scans the page for size guides
2. If a size-selector container is found, the **Inline Anchor Button** (`* DressApp size`) is mounted next to it
3. A **Floating Action Button (FAB)** (`* DressApp`) is also mounted in the bottom-left corner
4. Clicking either button triggers the automatic detection engine: Scans visible dialogs, description tabs, and parent layouts for HTML `<table>` containing size keywords, or large `<img>` matching chart aspect-ratios If found, captures a crop of the table or encodes the image to base64, querying the backend

Manual Crop Flow:

1. If autodetect fails, the interface shifts into **Crop Mode**
2. Semi-transparent dimming overlay covers the webpage with header: `Drag a box around the size chart`
3. **Drawing Gesture**: Left-click and hold, drag diagonally to outline the chart, release to lock

4. **Refining Layout**: Hovering over crop boundaries exposes 8 resize handles (edges and corners)
5. **Applying**: Click **Apply** (or press `Enter`) to confirm the crop
6. **Cancel**: Click **Cancel** (or press `Escape`) to exit crop mode

| State | Content | User Action | |---|---|---| | **Recommendation Card** | Recommended size, confidence index, reasoning text, matched columns, response latency, alternative size fits | Dismiss or retry | | **Warning Card** | Suspected typos in profile (e.g., `"Your shoulders (75 cm) look higher than expected"`) | Edit profile | | **Authentication Card** | Session expired/disconnected, link to `dressapp.co/extension/connect` | Login | | **Missing Measurements Card** | No circumferences or sizes registered in profile | Edit profile | | **No Matches Found** | Gemini reads chart but finds no row matching measurements | Retry with tighter crop | | **Orphaned Scripts** | Extension updated in background, invalidating content script port | Reload tab |

- **No Matches found (`No clear match`)**: Surfaces when Gemini reads the chart but finds no row matching the user's measurements. A **Retry** button allows re-cropping more tightly.
- **Orphaned Scripts (`DressApp was just updated`)**: Occurs when the Chrome extension updates in the background, invalidating the content script's port. Widget detects stale context and prompts reload.
- **Permission Requests**: If a screenshot fails because the extension lacks permissions, triggers browser permission dialog for optional `<all_urls>` permission.

| Resource | Limit | Rationale | |---|---|---| | `_ANALYZE_LOCK` | 1 concurrent request | Prevents Gemini quota contention | | Scroll retry | 3 attempts | Prevents infinite loops on short pages | | Lazy-image poll | 12s timeout, 600ms intervals | Balances completeness vs. speed | | Bookmarklet batch size | 15 cards per STREAM message | Memory management in postMessage | | Status cleanup | 60 seconds after completion | Prevents memory leak from job metadata |

3.12 Admin Panel Dashboard

System liveness validation, financial bookkeeping, and user account management.

1. Navigate to `/admin` (available to administrator roles).
2. **Overview**: Audit Gross Volumes and Platform Fee income summaries. Inspect the **Provider Activity Table** to view downstream liveness statistics (Gemini API, weather service latency, and error rates).
3. **Providers**: Click **Verify Key** to send a direct ping to the Gemini API. Toggle the **Eyes Vision Override** switch to route image analysis between the default Gemini endpoint and the on-premises `dressapp-eyes` (`Qwen 2.5 VL 3B`) container.
4. **Users**: View active credits, roles, and lifetime payments. Use direct actions to Promote or Demote users.
5. **Listings**: View listing states and toggle active flags to suspend fraudulent items.

3.13 Subscriptions, Billing, and Localized Payments

Integration with PayPal Subscriptions and Atzmai APIs for subscription management, credit purchases, and automated bookkeeping.

1. **Manager ($10/mo) & Professional ($15/mo) Subscriptions**: Upgrades Free Tier accounts (50–150 item ceiling, on-premises `Qwen 2.5 VL 3B` Eyes engine) to unlimited closet storage and fast Cloud AI routing (`Google Gemini 3.5 Flash`) via the PayPal Subscriptions REST API. Monthly and annual tiers with automatic recurring billing and instant webhooks updating user roles.
2. **Pre-Paid AI Credit Bucket Management**: **Daily Free Credits**: 10 free credits granted every 24 hours with a strict 30-day expiration window. **Paid Credit Packs**: Purchased globally via PayPal or locally via Atzmai; paid credits **never expire**: *10 credits pack*: $1.99 / 10.00 ILS *25 credits pack*: $3.99 / 25.00 ILS *50 credits pack*: $7.99 / 50.00 ILS *100 credits pack*: $15.99 / 55.00 ILS **Aging & Consumption Priority**: AI operations automatically consume credits from the oldest soonest-expiring free bucket first, protecting permanent paid credits.
3. **Atzmai Gateway Integration (Israel / ILS)**: Handles localized payments supporting regular credit cards, Bit payment direct checkout links, and standing orders.
4. **Webhook Processing & Invoice Delivery**: Authorized payments emit signed webhook notifications to `/api/v1/paypal/webhook` and `/api/v1/atzmai/webhook`. On verification, credits are credited atomically, and localized invoice PDFs are dispatched directly to the customer's email.

---

4. Expected Results

- **Ingestion**: Items immediately populate the closet grid (~16ms) across single, multi-garment, and mass-cluster photos. Background cutouts render clean, transparent PNG results.
- **DPP Verified Badge**: Scanning valid passports displays the green information card with sustainability details.
- **Avatar Outerwear**: Outerwear displays correctly layered over tops on the 2D avatar canvas without clipping headwear/shoes.
- **Voice Response**: Virtual Stylist text outputs play spoken audio automatically with a visible waveform indicator.
- **Subscriptions**: Activating a Manager or Professional plan immediately removes the closet limit warning and enables fast Cloud AI routing.

---

5. Troubleshooting

HTTP 402 Payment Required

- **Problem**: Ingestion blocked. You have reached the capacity limit (50 items baseline, up to 150 with referral slots).
- **Solution**: Go to the Pricing page (`/pricing`) and subscribe to the Manager ($10/mo) or Professional ($15/mo) plan, or share your invite link to get +10 slots per registration.

SSRF Blocked / DNS Error on DPP

- **Problem**: Scanned QR passport URL fails to parse.
- **Solution**: The parser blocks private IP addresses (e.g. `127.0.0.1`, `192.168.x.x`) to protect internal servers. Ensure QR codes point to public domains.

Camera / Microphone Permission Denied

- **Problem**: Capture/scan viewport displays an 'X' error screen, or voice typing fails.
- **Solution**: Open browser permissions, enable Camera and Microphone access for the domain, and reload.

Stylist Chat Quota Fallback ("Using Platform Stylist")

- **Problem**: A banner appears above the Stylist's answer: *"Using Platform Stylist (Quota Fallback)"*.
- **Explanation & Solution**: When upstream Google Gemini cloud rate limits (`429 Too Many Requests`) or transient quota spikes occur, DressApp's `FallbackBrain` seamlessly reroutes your request to the on-premises `dressapp-eyes` (`Qwen 2.5 VL 3B`) engine so your session never fails with a 500 error.

Out of Memory (OOM) VPS Spikes

- **Problem**: CPU/RAM peaks during upload processes.
- **Solution**: Ingestion utilizes sequential queue locks for batches >5 items. Ensure the server has at least 8 GB RAM (CPX32 configuration) to host `llama-server` (`Qwen 2.5 VL 3B`, ~2.8 GB RAM) alongside the FastAPI backend and SegFormer workers.

---

6. Limitations

- **Browser Web Speech APIs**: Native Speech-to-Text translation is restricted to Chrome and Safari; other browsers fall back to standard text input.
- **Offline Client Modulations**: Mobile offline Piper ONNX speech synthesis uses fewer voice profiles than the server-side Gemini audio modal.
- **Image Size Constraints**: Avatar and profile uploads are compressed locally in the browser to 82% quality to fit within the 16MB MongoDB document boundary.
- **Receipt Parsing Scope**: Highly blurred, distorted, or handwritten receipts may fail data extraction.